ACID Properties

Module Title and Code: Introduction to Databases, UFCFTK-30-1

Submitted By: Raunak Kumar Singh

Student Number: 24071246

Orbund Id: 11281

Submitted To: Rabin Gautam

TASK 4 (20%)

What are ACID properties in the context of database transactions? Explain with 1 example.

Answer:

ACID Properties are set of important principles used in the database to make sure that transactions work systematically and data stays intact. A transaction is basically a set of operations that are implemented together to get the task done.

The term **ACID** stands for:

A- Atomicity

C- Consistency

I- Isolation

D- Durability

These four properties help us to avoid errors and system crash when multiple operations are carried out in same time

Example: Typical Online Product Purchase

To explain all four properties, I have taken this example. Basically, when a customer purchases a product, this is how a system performs its operations:

1. **Reducing the product stock**
2. **Creating an order record**

```sql
BEGIN TRANSACTION;

UPDATE products
SET stock = stock - 1
WHERE product_id = 101;

INSERT INTO orders (product_id, user_id)
VALUES (101, 1);

COMMIT;
```

BEGIN TRANSACTION:

UPDATE products

SET stock = stock -1

WHERE product_id = 101;

INSERT INTO orders (product_id, user_id)

VALUES (101,1);

COMMIT:

A- Atomicity

Atomicity means a transaction is treated as a single unit. Either everything inside it happens, or nothing happens at all. There is no partial execution.

Application in this example:

Atomicity applies to the entire **TRANSACTION**,
from **BEGIN TRANSACTION** to **COMMIT**.
In order to maintain this property, both the operations should be executed (i.e. **Updating Stock** and **Inserting Order**) successfully.

If the programs fail after the **UPDATE** script but before the **INSERT** script, the **TRANSACTION** will be rolled back.

SQL Query:

```
ROLLBACK;
```

This way, the stock will not be reduced without an order being placed.

C-Consistency

Consistency ensures that the database always remains in a valid state and follows all well-defined rules and constraints.

Application in this example:

It applies during the implementation of the **UPDATE** script.

SQL Query:

```
UPDATE products SET stock = stock -1
```

The database makes sure that rule such as stock not becoming negative is maintained.

If the value of the Stock is already **0**, the **TRANSACTION** will not be executed further.

This way, the data stays valid before and after **TRANSACTION**.

I-Isolation

Isolation means that multiple transactions running at the same time should not interfere with each other.

Application in this example:

It applies while the **TRANSACTION** is being executed (Between **BEGIN TRANSACTION** and **COMMIT**).

If multiple users try to purchase the same last product at the same interval of time, the database ensures that only one **TRANSACTION** successfully updates the stock. Other one will be on hold or it will fail.

This prevents problems like selling same stuff twice. This is usually managed using locks or isolation levels.

D-Durability

Durability ensures that once a transaction is successfully completed, the changes are permanently stored in the system.

Application in this example:

It applies at the **COMMIT** stage.

SQL Query: **COMMIT;**

After the **TRANSACTION** is committed, both the updated stock and new order record are permanently stored.

Even if the system crashes immediately after the commit, the data will not be lost.

Conclusion

ACID Properties help ensure that transaction is done safely and correctly. Using the example of an **ONLINE PRODCUT PURCHASE**, we can see how each property works at different stages to keep the data accurate and reliable.

Tools and Technologies Used:

1. VS Code

I used VS Code to work on the coding part precisely to store **MySQL** Command to explain the **ACID Properties**.